

Tworzenie aplikacji webowych

Streamlit 3 (SQL)

STUDIA PODYPLOMOWE

Data Science w zastosowaniach biznesowych

Uniwersytet Warszawski, Wydział Nauk Ekonomicznych



UNIwersYTET
WARSAWski



WYDZIAŁ NAUK
EKONOMICZNYCH

Ewa Weychert, PhD

Copyright © 2025 WNE UW. All rights reserved.

Spis Treści

- 1 Podstawy sqlite3
- 2 Złączenia w SQLite
- 3 SQL + pandas
- 4 Podsumowanie

Dlaczego warto używać SQL w Pythonie?

- Połącz możliwości **Pythona** ze strukturą **SQL**.
- Python służy do:
 - czyszczenia i transformacji danych,
 - automatyzacji i skryptowania,
 - analizy i wizualizacji.
- SQL służy do:
 - odpytywania danych strukturalnych,
 - filtrowania, agregacji i złączeń,
 - pracy ze schematami relacyjnymi.
- Razem tworzą elastyczny i wydajny przepływ pracy z danymi.

SQLite w Pythonie: sqlite3

- **SQLite** to lekka, plikowa relacyjna baza danych.
- Python posiada wbudowany moduł: `sqlite3`.
- Świetny do:
 - małych i średnich zbiorów danych,
 - nauki i eksperymentowania,
 - lokalnych potoków analitycznych.
- Nie wymaga osobnego serwera: baza danych to jeden plik `.db`.

Łączenie z SQLite w Pythonie

```
import sqlite3

# Połącz się z bazą danych (lub utwórz plik)
conn = sqlite3.connect("example.db")

# Utwórz kursor do wykonywania poleceń SQL
cur = conn.cursor()
```

- `connect(...)` otwiera bazę danych.
- `cursor()` zwraca obiekt do wykonywania SQL.
- Pamiętaj, aby na końcu wywołać `conn.commit()` i `conn.close()`.

Tworzenie tabel

```
cur.execute("""  
CREATE TABLE IF NOT EXISTS users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    age INTEGER  
);  
""")  
conn.commit()
```

- CREATE TABLE IF NOT EXISTS zapobiega błędom.
- Używaj standardowego SQL DDL (języka definicji danych).

Wstawianie danych (parametryzowane)

```
users = [  
    ("Alice", 30),  
    ("Bob", 25),  
    ("Carol", 35)  
]  
  
cur.executemany("""  
INSERT INTO users (name, age)  
VALUES (?, ?);  
""", users)  
  
conn.commit()
```

- Używaj **symboli zastępczych** (?), aby uniknąć SQL injection.
- `executemany` efektywnie wstawia wiele wierszy.

Odpytywanie danych

```
cur.execute("SELECT id, name, age FROM users;")  
rows = cur.fetchall()
```

```
for row in rows:  
    print(row)
```

- `execute(...)` wysyła zapytanie SQL.
- `fetchall()` pobiera wszystkie wyniki jako listę krotek.

Złączenia relacyjne

- Złączenia łączą wiersze z dwóch tabel na podstawie pasujących kluczy.
- Najczęstsze typy:
 - **INNER JOIN** – tylko pasujące wiersze z obu tabel.
 - **LEFT JOIN** – wszystkie wiersze z lewej tabeli, pasujące z prawej.
 - **RIGHT JOIN** – wszystkie wiersze z prawej tabeli, pasujące z lewej.
 - **FULL OUTER JOIN** – wszystkie wiersze z obu, dopasowane gdy możliwe.
- SQLite natywnie obsługuje:
 - INNER JOIN
 - LEFT JOIN
- Pozostałe złączenia można *emulować*.

Przykład: dwie tabele

Tabela A

key	val_A
1	A1
2	A2
3	A3
4	A4

Tabela B

key	val_B
3	B3
4	B4a
4	B4b
5	B5

INNER JOIN w SQLite

```
SELECT A.key, A.val_A, B.val_B  
FROM A  
INNER JOIN B  
    ON A.key = B.key;
```

- Zwraca tylko wiersze, gdzie $A.key = B.key$.
- Klucze obecne tylko w jednej tabeli są pomijane.

LEFT JOIN w SQLite

```
SELECT A.key, A.val_A, B.val_B  
FROM A  
LEFT JOIN B  
    ON A.key = B.key;
```

- Wszystkie wiersze z A, pasujące wiersze z B.
- Niepasujące wiersze z B przyjmują wartość NULL.

Emulowanie FULL OUTER JOIN w SQLite

```
SELECT A.key, A.val_A, B.val_B  
FROM A  
LEFT JOIN B ON A.key = B.key
```

UNION

```
SELECT B.key, A.val_A, B.val_B  
FROM B  
LEFT JOIN A ON B.key = A.key  
WHERE A.key IS NULL;
```

- Łączy:
 - wszystkie wiersze z A (left join),
 - oraz wiersze z B, które nie pasują do A.

Łączenie SQL z pandas

- Często chcemy:
 - Wykonać zapytania SQL w bazie danych.
 - Załadować wyniki do `pandas.DataFrame`.
 - Kontynuować analizę przy użyciu pandas.
- pandas udostępnia wygodne funkcje pomocnicze:
 - `pd.read_sql_query(...)`
 - `pd.read_sql_table(...)`

Wczytywanie wyników SQL do pandas

```
import pandas as pd
import sqlite3

conn = sqlite3.connect("example.db")

df_users = pd.read_sql_query(
    "SELECT id, name, age FROM users;",
    conn
)

print(df_users)
conn.close()
```

- `read_sql_query` wykonuje SQL i zwraca `DataFrame`.

Złączenia w pandas za pomocą merge()

```
import pandas as pd
```

```
A = pd.DataFrame({  
    "key":    [1, 2, 3, 4],  
    "val_A": ["A1", "A2", "A3", "A4"]  
})
```

```
B = pd.DataFrame({  
    "key":    [3, 4, 4, 5],  
    "val_B": ["B3", "B4a", "B4b", "B5"]  
})
```

```
inner_join = A.merge(B, on="key", how="inner")
```

- `how="inner"` odpowiada SQL INNER JOIN.
- Inne opcje: `"left"`, `"right"`, `"outer"`.

Anti-Join w pandas

```
left_anti = (  
    A.merge(B, on="key", how="left", indicator=True)  
    .query("_merge == 'left_only'")  
    .drop(columns="_merge")  
)
```

- Wybiera wiersze, które istnieją *tylko* w A.
- Ten wzorec jest przydatny do:
 - walidacji danych,
 - znajdowania niedopasowanych rekordów,
 - implementowania różnic zbiorów.

Porównanie złączeń SQL i merge() w pandas

- Konceptualnie te same operacje:
 - dopasowywanie wierszy na podstawie kluczy,
 - kontrola, które niedopasowane wiersze są zachowywane.
- SQL:
 - Deklaratywny: opisuje *jaki* wynik chcemy uzyskać.
 - Świetny do baz danych i dużych zbiorów na dysku.
- pandas:
 - Imperatywny/programowalny: część kodu Pythona.
 - Świetny do analizy w pamięci i szybkich eksperymentów.

Podsumowanie

- Python płynnie integruje się z SQL poprzez `sqlite3` i inne biblioteki.
- SQLite jest idealny do lekkich, lokalnych relacyjnych baz danych.
- Złączenia SQL i `merge()` w pandas wyrażają te same idee relacyjne różną składnią.
- pandas potrafi zarówno:
 - *konsumować* wyniki zapytań SQL,
 - jak i wykonywać dodatkowe złączenia i transformacje.
- Znajomość zarówno SQL, jak i pandas daje elastyczne możliwości pracy z danymi.

Dlaczego SQL jest przydatny w aplikacji webowej

1 Dlaczego SQL wewnątrz aplikacji Streamlit?

Bez bazy danych aplikacja Streamlit jest zasadniczo „zapominalska”:

- Informacje przechowywane tylko w zmiennych znikają po odświeżeniu lub ponownym uruchomieniu.
- Wielu użytkowników nie może współdzielić ani utrzymywać stanu.

Z SQLite + Streamlit:

- Dane wprowadzone przez użytkownika (np. formularze) są zapisywane w trwałym pliku bazy danych.
- Dane można później odpytywać, filtrować, grupować lub agregować za pomocą SQL.
- Wyniki można wczytać do pandas w celu dalszej analizy lub wizualizacji.
- Konfiguracja jest bardzo prosta i nie wymaga osobnego serwera bazy danych.

Streamlit zapewnia interfejs; SQLite zapewnia pamięć i strukturę.

2 Podstawowa architektura

Konceptualnie:

- `app.py` (Streamlit):
 - Wyświetla elementy UI, takie jak formularze i przyciski
 - Wywołuje funkcje pomocnicze komunikujące się z bazą danych
- `my_database.db` (plik SQLite):
 - Przechowuje tabele relacyjne (np. pracownicy)
 - Zapisuje i pobiera dane za pomocą operacji SQL

Zamiast ręcznie pisać kod Python, użytkownicy korzystają z formularzy, a SQL obsługuje warstwę trwałości.

Dlaczego SQL jest przydatny w aplikacji webowej

Zbieranie danych od użytkownika za pomocą formularza:

- Interfejs zawiera pola tekstowe, listy rozwijane i pola liczbowe.
- Po zapisaniu pracownika aplikacja przekazuje dane do funkcji pomocniczej bazy danych.

Utrwalanie danych za pomocą SQLite:

- Funkcja pomocnicza zapisuje przesłane informacje w bazie danych.
- Dzięki temu dane pozostają dostępne po restarcie lub odświeżeniu.

Wyświetlanie i filtrowanie danych:

- Aplikacja pobiera dane z bazy i wyświetla je w interaktywnej tabeli.
- Filtrowanie według działu lub innych atrybutów to proste zapytanie SQL.

Łatwe rozszerzanie:

- Dodaj akcje edycji/usuwania dla każdego pracownika.
- Dodaj więcej powiązanych tabel i łącz je złączeniami.
- Dodaj wykresy podsumowujące obliczone z agregacji SQL.

Kolejne kroki:

- Umożliwienie edytowania i usuwania pracowników bezpośrednio z poziomu interfejsu.
- Budowanie analogicznych aplikacji dla swoich przykładów z zakresu sądownictwa lub leśnictwa.

Co robi app_sql_1.py

- **Logowanie / Rejestracja** Dane uwierzytelniające są przechowywane w prostej tabeli `users`.
- **Dashboard** Wyświetla kluczowe metryki obliczone na podstawie tabeli `user_actions`.
- **Rejestrowanie kliknięć** Każde kliknięcie (np. polubienie dashboardu lub pobranie raportu) dodaje wiersz do `user_actions` z sygnaturą czasową.
- **Widoki historii**
 - Osobisty dziennik aktywności filtrowany według nazwy użytkownika.
 - Systemowy dziennik aktywności do celów administracyjnych.

Ta struktura może być dostosowana do własnej dziedziny — np. spraw sądowych lub inwentaryzacji leśnej.

Cel aplikacji

- Zbuduj interaktywną aplikację **Streamlit** obsługującą:
 - Uwierzytelnianie użytkowników (logowanie i rejestracja)
 - Rejestrowanie działań użytkowników w SQL
 - Eksploracyjną analizę danych (EDA) dla wczytanych plików CSV
- Użyj małej bazy **SQLite** (app.db) do przechowywania:
 - Danych kont użytkowników
 - Logów działań użytkowników
- Zapewnij ustrukturyzowany interfejs EDA z:
 - Panelem przeglądu
 - Inspekcją brakujących danych i typów
 - Analizą zmiennych numerycznych
 - Analizą zmiennych kategoriycznych

Architektura wysokiego poziomu

- **Frontend Streamlit**

- Interfejs logowania / rejestracji
- Wielozakładowy eksplorator danych

- **Backend SQLite**

- Przechowuje konta użytkowników
- Przechowuje wszystkie zarejestrowane interakcje użytkowników
- Opcjonalna kolumna szczegółów dla dodatkowych metadanych

- **Warstwa EDA**

- Używa pandas do obsługi danych
- Używa matplotlib do wizualizacji
- Zorganizowana w 4 zakładki, każda z 4 logicznymi sekcjami

Warstwa SQL / uwierzytelniania: ogólny przegląd

- Aplikacja otwiera połączenie z lokalnym plikiem bazy SQLite.
- Przy uruchomieniu baza danych jest inicjalizowana:
 - Tworzona jest tabela użytkowników, jeśli nie istnieje.
 - Tworzona jest tabela działań użytkowników, jeśli nie istnieje.
 - Starsze bazy są aktualizowane o nowsze pola (np. kolumna szczegółów).
- Tabela **users** zawiera:
 - unikalną nazwę użytkownika,
 - hasło (tylko do celów demonstracyjnych — bez haszowania),
 - automatycznie inkrementowane ID.
- Tabela **user_actions** śledzi:
 - kto co zrobił,
 - kiedy to zrobił,
 - opcjonalne dodatkowe informacje.

Funkcje pomocnicze uwierzytelniania (konceptualnie)

- Funkcja tworzy nowe konto użytkownika, zapewniając unikalność nazw.
- Funkcja weryfikacji sprawdza, czy nazwa użytkownika i hasło pasują do istniejącego rekordu.
- Te funkcje pomocnicze pozwalają Streamlit przełączać się między stanami zalogowanym i wylogowanym.

Rejestrowanie działań użytkowników (konceptualnie)

- W całej aplikacji działania użytkowników są rejestrowane w bazie danych.
- Przykłady obejmują:
 - pomyślne logowania,
 - przesyłanie plików,
 - zapisywanie migawek analizy,
 - przełączanie zakładek lub naciskanie przycisków.
- Tworzy to pełny ślad audytowy zachowania użytkowników do późniejszej analizy.

Ekran logowania / rejestracji

- Strona logowania zawiera dwie zakładki: jedną do logowania, drugą do rejestracji.
- Pomyślne logowanie aktualizuje stan sesji i rejestruje zdarzenie.
- Zakładka rejestracji sprawdza dostępność nazwy użytkownika i dostarcza informację zwrotną.

Układ EDA: główna idea

- Po zalogowaniu użytkownik widzi **Eksplorator danych**.
- Użytkownik wgrywa plik CSV, który jest analizowany w czterech zakładkach:
 - ➊ Przegląd
 - ➋ Braki i typy
 - ➌ Zmienne ciągłe
 - ➍ Zmienne kategoryczne
- Każda zakładka używa czteroblokowego układu dla ustrukturyzowanego wyświetlania.
- Kontrolki paska bocznego pozwalają dostosować:
 - liczbę wierszy do podglądu,
 - liczbę przedziałów histogramu,
 - liczbę kategorii do wyświetlenia.

Panele EDA (opisy konceptualne)

Panel przeglądu

- Pokazuje kształt zestawu danych, pierwsze wiersze i nazwy kolumn.
- Dostarcza podsumowania numeryczne i katagoryczne.

Brakujące wartości i typy danych

- Wyświetla liczby i procenty brakujących wartości.
- Pokazuje typy danych i liczbę unikalnych wartości.

Zmienne ciągłe

- Histogramy generowane dla wszystkich kolumn numerycznych.
- Wyświetlanie w układzie dwukolumnowym.

Zmienne katagoryczne

- Wykresy słupkowe dla top-N kategorii w każdej kolumnie.
- Obrócone etykiety dla czytelności.

Główny przepływ aplikacji

- Przy uruchomieniu tabele bazy danych są tworzone lub aktualizowane.
- Pasek boczny wskazuje status logowania i zawiera przycisk wylogowania.
- Główny obszar wyświetla:
 - stronę logowania/rejestracji, jeśli żaden użytkownik nie jest zalogowany,
 - pełny dashboard EDA, jeśli użytkownik jest uwierzytelniony.
- Wszystkie ważne interakcje rejestrują odpowiednie zdarzenie w bazie danych.

Podsumowanie

- Aplikacja demonstruje, jak:
 - połączyć **Streamlit** z **SQLite**,
 - zbudować minimalny system uwierzytelniania,
 - śledzić zachowanie użytkowników za pomocą logów SQL,
 - stworzyć ustrukturyzowany, zakładkowy przepływ pracy EDA.
- Ta architektura może być rozszerzona o:
 - bardziej zaawansowane uwierzytelnianie,
 - bogatsze rejestrowanie,
 - dashboardy dziedzinowe (np. sądownictwo, leśnictwo, finanse).